

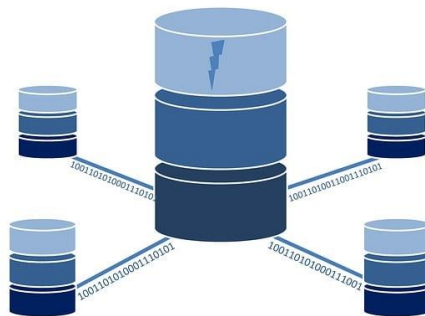
Universidad Tecnológica Nacional

Facultad Regional General Pacheco



TUP - Base de Datos II

Trabajo Práctico Integrador



Equipo 04

Integrantes:

Dieser, Germán	Legajo: 30957
Fredes, Jacob Samuel	Legajo: 27233
Frias, Tamara Ruth	Legajo: 24973
Hiebl, Darian	Legajo: 30390

Explicación del sistema:

El sistema será una aplicación web para la gestión, interacción y reproducción musical. La base de datos almacenará información sobre usuarios, canciones, artistas, álbumes, géneros, discográficas y tipos de contenido. Los usuarios podrán registrarse, buscar música, crear playlists, marcar canciones como favoritas y consultar letras con traducción. También interactuar con comentarios, seguir artistas u otros usuarios. El sistema permitirá a los artistas y productoras subir contenido, gestionar sus perfiles y acceder a estadísticas de reproducciones y actividad.

Funcionalidades principales:

Los usuarios podrán:

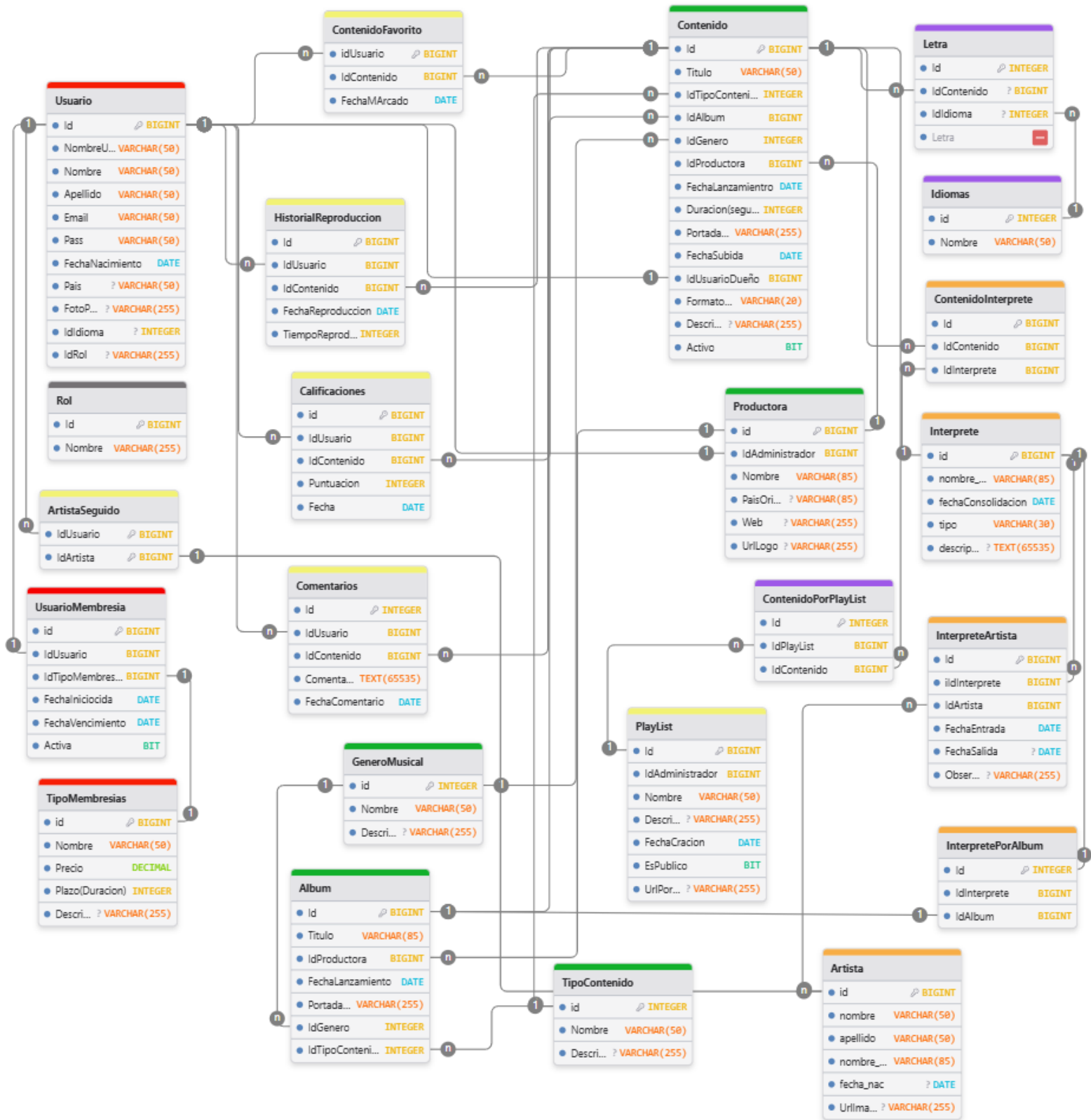
- Buscar acceder a contenido, sus género, autor, año, álbum o discográfica.
- Gestionar datos de usuario, seguimiento de artistas y usuarios, historial de reproducción, calificaciones, comentarios, tipo de membresía (gratuita o por suscripción, con opciones mensuales con diferentes características).
- Marcar canciones como favoritas y crear playlists personalizadas.
- Consultar letras y sus traducciones según el idioma elegido.
- Visualizar rankings de contenido más escuchado.
- Acceder a estadísticas de reproducciones.

El sistema contará además con un portal para artistas y productoras, donde podrán:

- Subir contenido (música, podcasts, audiolibros, etc.) y gestionar su perfil.
- Cargar letras, formatos, datos relevantes y comentarios sobre sus obras.
- Acceder a estadísticas de reproducciones, likes y actividad.

El sistema está construido con una base de datos relacional que almacenará información sobre usuarios, canciones, autores, intérpretes, álbumes, géneros, discográficas, accesos y tipos de contenido, lo cual permitirá personalizar la experiencia y mejorar la gestión general de la plataforma.

Diagrama de Entidad Relación:



Objetos de Base de Datos clave en el Sistema:

Vista 1:

Se utiliza la vista **vista_topNCanciones** que muestra las canciones activas más reproducidas en orden descendente según la cantidad total de reproducciones. Esta vista está destinada a ser consultada por administradores, y contiene la siguiente información: ID de la canción, título, fecha de lanzamiento, duración, descripción, nombre del género y total de reproducciones registradas.

```
CREATE VIEW vista_topNCanciones AS
SELECT TOP 100
    c.Id AS id_cancion,
    c.Titulo,
    c.FechaLanzamiento,
    c.Duracion,
    c.Descripcion,
    g.Nombre AS Genero,
    c.ContadorReproducciones AS TotalReproducciones
FROM Contenido c
INNER JOIN GeneroMusical g ON g.Id = c.IdGenero
WHERE c.Activo = 1
ORDER BY
    c.ContadorReproducciones DESC;
```

Vista 2:

Se utiliza la vista **vista_usuariosActivos** que muestra los usuarios con suscripciones activas. Esta vista está destinada a ser consultada por administradores o personal del sistema para realizar seguimiento de los miembros vigentes. Contiene la siguiente información: ID del usuario, nombre de usuario, nombre, apellido, correo electrónico y fecha de vencimiento de la membresía activa.

```
CREATE VIEW vista_usuariosActivos AS
SELECT
    U.Id AS IdUsuario,
    U.NombreUsuario,
    U.Nombre,
    U.Apellido,
    U.Email,
    UM.FechaVencimiento
FROM Usuario U INNER JOIN UsuarioMembresia um ON UM.IdUsuario = U.Id WHERE UM.Activa = 1;
```

Vista 3:

Se utiliza la vista **vista_comentariosRecientes** para mostrar los comentarios más recientes de los usuarios sobre cualquier tipo de contenido (canciones, podcasts, etc.)

Nos permite ver el nombre del usuario, el nombre del contenido, el comentario y la fecha en que se realizó el mismo.

```
CREATE VIEW vista_comentariosRecientes AS
SELECT
  c.Id AS IdComentario,
  c.IdContenido,
  u.NombreUsuario,
  ct.Titulo AS Contenido,
  c.Comentario,
  c.FechaComentario
FROM Comentarios c
JOIN Usuario u ON c.IdUsuario = u.Id
JOIN Contenido ct ON c.IdContenido = ct.Id;
```

Procedimiento Almacenado 1:

Se utiliza el procedimiento almacenado **sp_eliminarPlaylist** para eliminar una playlist específica del sistema junto con todas sus asociaciones en la tabla intermedia de contenidos por playlist. El mismo recibe como parámetro el ID de la playlist a eliminar y garantiza la integridad mediante una transacción. Está destinado a ser utilizado por administradores o usuarios con permisos de edición sobre las listas.

```
CREATE PROCEDURE sp_eliminarPlaylist
  @IdPlaylist BIGINT
AS
BEGIN
  BEGIN TRY
    BEGIN TRANSACTION;
    DELETE FROM ContenidoPorPlaylist
    WHERE IdPlaylist = @IdPlaylist;
    DELETE FROM Playlist
    WHERE Id = @IdPlaylist;

    COMMIT TRANSACTION;
    PRINT 'Playlist eliminada';
  END TRY
  BEGIN CATCH
    ROLLBACK TRANSACTION;

    PRINT 'Error durante la eliminacion: ' + ERROR_MESSAGE();
  END CATCH
END;
```

Procedimiento Almacenado 2:

Se utiliza el procedimiento almacenado **sp_agregarCancionAPlaylist** para agregar una canción a una playlist siempre que no se encuentre previamente ahí.

Verifica si la canción se encuentra en la playlist para no repetirla, devuelve un mensaje si ya se encuentra en la playlist, en caso contrario la agrega.

```
CREATE PROCEDURE sp_agregarCancionAPlaylist
    @IdPlaylist BIGINT,
    @IdContenido BIGINT
AS
BEGIN
    IF EXISTS (
        SELECT IdContenido
        FROM ContenidoPorPlaylist
        WHERE IdPlaylist = @IdPlaylist AND IdContenido = @IdContenido
    )
    BEGIN
        PRINT 'Este contenido ya esta en la lista.';
        RETURN;
    END

    INSERT INTO ContenidoPorPlaylist (IdPlaylist, IdContenido)
    VALUES (@IdPlaylist, @IdContenido);

    PRINT 'Contenido agregado correctamente a la list.';
END;
```

Trigger 1:

Se utiliza el **trigger trg_finMembresia** para actualizar automáticamente el estado de la membresía de los usuarios cuando llega su fecha de vencimiento. Este trigger se ejecuta después de cualquier inserción o actualización sobre la tabla UsuarioMembresia, desactivando aquellas membresías cuya fecha de vencimiento haya pasado y aún estén activas.

```
CREATE TRIGGER trg_finMembresia
ON UsuarioMembresia
AFTER INSERT, UPDATE
AS
BEGIN

    UPDATE UsuarioMembresia
    SET Activa = 0
    WHERE FechaVencimiento < GETDATE()
    AND Activa = 1;

    PRINT 'Se actualizo con exito.';
END;
```

Trigger 2:

Se utiliza el trigger **trg_registrarReproduccion** para contar la cantidad de reproducciones que posee una canción.

Para poder hacer este reporte se requirió agregar una columna, para llevar el registro de la cantidad de reproducción que tiene la canción, en la tabla Contenido.

```
CREATE TRIGGER trg_registrarReproduccion
ON HistorialReproduccion
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE c
    SET c.ContadorReproducciones = c.ContadorReproducciones + r.cant
    FROM Contenido c
    INNER JOIN (
        SELECT IdContenido, COUNT(*) AS cant
        FROM inserted
        GROUP BY IdContenido
    ) r ON c.Id = r.IdContenido;
END;
```

Trigger 3:

Se utiliza el trigger **trg_limitarPuntuacion** para evitar que se vuelva a calificar un contenido evitando duplicado de puntuación en la tabla Calificaciones o que se puntúe más de una vez el mismo contenido. Y así no afectar al sistema de puntajes.

Éste trigger se dispara antes de un insert en la tabla Calificaciones. Valida si un determinado usuario ya ha calificado el contenido. Si ya existe una calificación de ese usuario para un contenido determinado, detiene la inserción y devuelve un mensaje, sino existe, lo inserta en la tabla Calificaciones.

```

CREATE TRIGGER trg_limitarPuntuacion
ON Calificaciones
INSTEAD OF INSERT
AS
BEGIN
    IF EXISTS(
        SELECT 1
        FROM Calificaciones c
        JOIN inserted i ON c.IdUsuario = i.IdUsuario AND c.IdContenido = i.IdContenido
    )
    BEGIN
        PRINT 'Ya existe una calificación para este contenido.'
        RETURN;
    END

    INSERT INTO Calificaciones(IdUsuario, IdContenido, Puntuacion, Fecha)
    SELECT IdUsuario, IdContenido, Puntuacion, Fecha
    FROM inserted;
END;

```

Función 1:

Se utiliza la función fn_duracionTotalPlaylist para calcular la duración total en segundos del contenido incluido en una playlist determinada. Esta función recibe como parámetro el ID de la playlist y devuelve un valor entero con la suma de todas las duraciones de los contenidos asociados.

```

CREATE FUNCTION fn_duracionTotalPlaylist (@IdPlaylist BIGINT)
RETURNS INT
AS
BEGIN
    DECLARE @DuracionTotal INT;

    SELECT @DuracionTotal = SUM(c.duracion)
    FROM ContenidoPorPlaylist cp
    INNER JOIN Contenido c ON c.id = cp.IdContenido
    WHERE cp.IdPlaylist = @IdPlaylist;

    RETURN ISNULL(@DuracionTotal, 0);
END;

```


Links a los recursos

Script de creación de base de datos con datos

<https://github.com/GDieser/BD2-TPI-G04>

Video demo del sistema

<https://drive.google.com/file/d/1-wnvZJDPwOaCh2BrDxGdLWUPBYLHyprw/view?usp=sharing>